

Correcting lineshape distortions using ShimNet

Krzysztof Kazimierczuk , Sylwia Jopa, Marek Bukowicki

ShimNet is a neural network for post-acquisition correction of NMR spectra distorted by magnetic field inhomogeneity. The network is trained using synthetic spectra convolved with “natural” distortions measured in a calibration procedure. The software is described in the paper:

The hands-on will cover:

1. The use of ShimNet program and a web service.
2. The collection of calibration spectra and the extraction of distortions
3. The generation of training spectra
4. The example neural network training
5. The comparison of networks obtained using different training parameters.

The ShimNet is a standalone Python script, generally run from the command line. However, for better understanding and consistent visualization of the results, we will work today in the Google Colab environment.

To start, log in to Google Colab and type:

```
from google.colab import drive
drive.mount('/content/drive')
%cd /content
!git clone -b workshops https://github.com/center4ml/shimnet.git
!mv shimnet /content/drive/MyDrive/shimnet_workshop
```

This will connect Colab to your Google Drive (login needed) and download the ShimNet package with example data from GitHub.

Open the notebook named *ShimNet_workshop1* from your Google Drive (folder *shimnet_workshop/Workshop_Colab_Notebooks*).

Keep Google Drive open in a separate tab or window.

Run the first cell to enter the *shimnet_workshop* folder, install all the required Python packages, and download the data.

1. The use of ShimNet program and webservice.

The cell named *ShimNet Predict* contains the command starting the ShimNet reconstruction. The following arguments are provided to the program:

- input (distorted) spectrum (text file with two columns)
- model (trained network)
- config file
- output folder

***Run the cell to correct the distorted spectrum using ShimNet (set proper input).
Run the next cell named Plot results (set the proper variable names).***

The input argument can be changed to run ShimNet on the other spectra (listed below). For some spectra the model and config file has to be changed from 600 MHz to 700 MHz.

`"Asarone_deshimmed"`

Spectrum of 14 mM Asarone in MeOD (intentionally) deshimmed using Z1 and Z2 shims

`"Asarone_reduce_volume"`

Sample shimmed, then removed from the magnet, 20 ul subtracted, sample inserted again and spectrum re-acquired without shimming

`"lineshape_deshimmed"`

Spectrum of 1% CHCl₃ in acetone-d (intentionally) deshimmed using Z1 and Z2 shims

`"multi_singlets_deshimmed"`

Spectrum of a mixture giving several singlets in a ¹H NMR spectrum.

`"Cresol_Red_after_different_sample"`

Spectrum of 0.78 mM Cresol Red in (wet) DMSO with huge H₂O peak present. Shims adjusted using different sample. The range of signal intensities is very high, rarely present in the training data. Thus, the correction is suboptimal. Compare with the next spectrum.

`"Cresol_Red_after_different_sample_cut"`

Spectrum of 0.78 mM Cresol Red in (wet) DMSO with huge H₂O peak zeroed. Shims adjusted using different sample.

`"styrene_1mm_600MHz"`

Spectrum of 3 M styrene in CDCl₃. Field homogeneity distorted by vertically shifting the sample by 1 mm in the spinner.

`"2etnaft_600MHz_deshimmed"`

Spectrum of 3 M 2-ethylnaphthalene in CDCl₃. Wrongly shimmed in Z1 and Z2.

Open the web browser and go to

<https://huggingface.co/spaces/NMR-CeNT-UW/ShimNet>

Upload selected dataset and run the processing.

2. The collection of calibration spectra and the extraction of distortions

ShimNet requires the collection of calibration spectra acquired in inhomogeneous magnetic fields with various spatial profiles. These are generated by missetting the Z1 and Z2 shim coil currents in a controlled way. Then, the distortion functions are deconvolved from the calibration spectra and used to generate the training data for the neural network.

This notebook demonstrates:

- how the calibration data is collected
- the idea of convolution and deconvolution
- how shim coil response functions are extracted from the calibration data

Open the Colab notebook named ShimNet_workshop2

Run the cell named Environment setup to connect to Google Drive and install packages

Run the second cell to see the Python script for acquiring the calibration data on the Agilent machine.

The third cell demonstrates the concept of convolution. Run it to see how the simulated spectrum is distorted by a “triangular” function.

In the next cell we try to determine the distortion function from the distorted spectrum by fitting it with the convolution of the original spectrum and modelled distortion. Run it and check the plots. First plot shows how the distortion function is recovered. The second plot shows how the convolution of recovered distortion and original spectrum matches the distorted spectrum. Change the fitting window (`response_function_estimate_length`) window and check the effect.

In the next cell we repeat the calculations for the noisy spectrum. Change the noise levels to see the effect on the fit.

In the fourth cell we use only a part of a spectrum to fit the distortion. This is the procedure used in Reference Deconvolution method - we fit only one, well-resolved peak whose “original” lineshape can be safely assumed.

In the last cell, we will extract the experimental shim coil response functions (SCRFs). Run it to see the plots of example CHCl₃ peaks and deconvolved distortion functions. Deconvolved functions are stored in a file and will be used in the next step to generate training spectra. The extraction script uses the following parameters:

```
# input
data_dir - folder with raw FIDs from the calibration loop
opti_fid_path = "/opti_spectra/opti_total6.fid" - folder with raw FID of optimally shimmed calibration sample

# output
```

`spectra_file = "./sample_run/total.npy"` - file with parts of spectra used for deconvolution
`spectra_file_names = "./sample_run/total.csv"` - file with names of fids from calibration
`opti_spectrum_file = "./sample_run/opti.npy"` - file with part of opti spectrum used for calibration
`responses_file = "./sample_run/scrf_shimotor9.pt"` - extracted distortion functions
`losses_file = "./sample_run/losses_scrf_shimotor9.pt"` - file with loss values of respective distortion functions deconvolution

3. The generation of training spectra

ShimNet is trained using thousands or millions of simulated spectra generated using:

- experimental distortion functions extracted from the calibration spectra (above)
- multiplets patterns from the text file
(the format of Ambit NMR software, see <https://zenodo.org/records/4629851>).
Generated from ENAMINE database with typical small molecules
(<https://enamine.net/compound-collections/real-compounds/real-database>).
- parameters from the config.yaml file

Normally, the generation of training data and actual network training are realized within one script. For demonstration purposes, we split them here.

Open the notebook named ShimNet_workshop3.

Run the first cell to mount Google Drive and install required packages.

Run the second cell to display the config file used to set up the parameters of the generated training spectra.

The parameters have the following meaning:

`pixels` - number of points in the spectrum
`frq_step` - frequency step size [Hz]
`relative_frequency_min and _max` - range of the spectrum where multiplets are located (relative)
`number_of_signals_min and _max` - number of multiplets in the spectrum
`thf_min and _max` - spectrum height component range
`relative_height_min and _max` - multiplet height component range
`spectrum_width_min and _max` - spectrum width component range [Hz]
`relative_width_min and _max` - multiplet width component range
`trf_min and _max` - Gaussian fraction in multiplet shape
`multiplicity_j1_min and _max` - A-X couplings
`multiplicity_j2_min and _max` - A-Y couplings
`spectrum_noise_min and _max` - Gaussian noise added

Run the next cell to generate the batch of 10 spectra according to the config file.

Run the cell named *Display spectra to show the results*. Change the `spectrum_index` to view various spectra in the batch.

In the next cell we will modify the data generation parameters (config) and check the results. We use only “data” part of the config file. Play with the parameters and check the results.

In the next cells you can use files with different multiplet patterns:

`data/singlets.txt`

`data/singlets_doublets_triplets.txt`

4. The example neural network training

Now, we will run the actual network training. For the sake of time, it will be very short training, insufficient to generate an optimal network.

Open the notebook named *ShimNet_workshop4*.

Run the first cell to mount Google Drive and install required packages.

Change environment to GPU (Runtime -> Change Runtime Type -> T4 GPU)

The second cell contains the full config file, whose ‘data’ section was used in the previous exercise. Run it to create `training_1` folder with `config.yaml` file.

The third cell runs the neural network training. The results are stored in `./my_model/training_1`. Run the cell and see the output and files appearing in the output folder. The log shows the batch number, loss function,

5. The comparison of networks obtained using different training parameters.